

Indian Institute of Information Technology, Una



AnyChat: Local Retrieval-Augmented Generation

Presented by: Yash Rana
(25MTCS017)



Outline

- Introduction
- What is RAG?
- Core Technologies (FAISS & LangChain)
- Impact of Top-K
- What is AnyChat?
- Target Use Cases
- How it Works Architecture
- Process Flowchart
- Evaluation Results
- Future Scope
- Conclusion



Introduction

- **The Problem:** Standard LLMs lack specific, or real-time data and can suffer from regular "hallucinations".
- **The Traditional Block:** Fine-tuning base models on custom data is computationally expensive, slow, and permanently bakes rigid memory into the model.
- **The Modern Solution:** Injecting private, targeted context into the model's working memory at runtime, guiding the model dynamically without altering its core weights.



What is RAG?

- **RAG** stands for **R**etrieval-**A**ugmented **G**eneration.
- **Retrieval:** Searching a localized, private database for information mathematically relevant to the user's prompt.
- **Augmentation:** Fusing those retrieved data chunks into the LLM's system prompt invisibly.
- **Generation:** Instructing the LLM to formulate an answer **only** based explicitly on that context.
- **Benefit:** Eliminates hallucination by forcing the model to cite its contextual sources.



What is FAISS?

Facebook AI Similarity Search.

- An extremely optimized, open-source library built for the rapid similarity search and clustering of dense vectors.
- When documents are broken down into coordinates (embeddings) indicating their semantic definition, FAISS instantly locates which data coordinates are geometrically closest to the question being asked.



What 'K' Does

In vector retrieval, **Top-K** represents the **number of nearest neighbor chunks** fetched from the FAISS index.

- **Low K (e.g., 2):** Highly strict and focused. Guarantees tight accuracy but risks missing the broader, overarching context.
- **High K (e.g., 8+):** Gives the model a massive amount of surrounding context. Risks confusing the LLM with loosely related, noisy details.

The Key: Balancing K is crucial to optimizing RAG accuracy limits dependent upon LLM context window sizes.



What is LangChain?

- **LangChain** is a framework used to build applications that combine LLMs with external data sources.
- Provides the standard, robust connective architecture for:
 - Document Loading (PDFs, Web crawling pipelines).
 - Text Splitting (Recursive chunking algorithms).
 - Can implement routing across multiple retrievers.
 - Streamlined LLM chain prompting protocols.



What is AnyChat?

- A locally-run, RAG web application built with a modern "**NotebookLM**" inspired application.
- Accepts multi-modal text injections: Seamlessly integrates **PDFs**, **DOCX**, **TXT**, and dynamic **Website URLs** recursively.
- **Completely local:** Uses CPU embeddings via `sentence-transformers` and the compact `TinyLlama` model, guaranteeing zero data leakage to cloud providers.



AnyChat Screenshots:

AnyChat

About

Sources

+ Add sources

Appsec.pdf

ARS.pdf

ForcedBytecodePatching.pdf

SOURCE STATS

112 Chunks Extracted

7.3 MB Size

l.ntu.edu.sg/el...

Chat

Multi-K Evaluation Context

K = 1

Static malware analysis is a technique that analyzes the code and functionality of an application without executing it. This approach relies on identifying known patterns and behaviors within the program, which can be used to identify potential vulnerabilities or weaknesses.

On the other hand, dynamic malware analysis involves running the software under test in real-time to monitor its behavior and performance, which helps in detecting malicious activities such as stealing sensitive data, accessing protected resources, or disabling antivirus software.

In static analysis, the code is first compiled into machine code, which may affect the execution speed and memory usage. However, this approach ensures that the software will perform correctly at runtime, but requires additional overhead and processing power. In contrast, dynamic analysis can provide more accurate results due to the availability of the operating system's virtualization technology to run the code in a simulated environment.

This distinction can help security professionals determine the TOP-K 1,3,5 each when analyzing malicious software, taking into account factors such as resource constraints, budget

Explain the difference between static and dynamic malware analysis.

+ Ask anything

→

Evaluation Studio

K = 1

K = 3

K = 5

Context

Metrics

SOURCES DISTRIBUTION

ARS.pdf

ForcedBytecodePatching.pdf

AVERAGE SIMILARITY

0.63

How it's calculated:

Mean Cosine Similarity (0 to 1) of all retrieved chunks.

Why it's relevant:

High scores mean chunks are semantically very close to your exact query subject.

FAITHFULNESS

0.67

How it's calculated:

Ratio of words in the LLM answer that directly overlap with words in the retrieved chunks.

Why it's relevant:

Low faithfulness implies the LLM might be hallucinating outside the loaded context.

RELEVANCE

100%

How it's calculated:

Percentage of retrieved chunks passing a strict similarity threshold (≥ 0.61).

Why it's relevant:

Indicates if the retrieval hit high-quality paragraphs or just loosely related filler text.

19-09-2026

9



AnyChat Features:

Sources

+ Add sources

☐

Appsec.pdf

☐

ARS.pdf

☐

ForcedBytecodePatching.pdf

☐

<https://personal.ntu.edu.sg/el...>

Add Knowledge

Website URL

Crawl Depth

Back

Crawl

K = 1 K = 3 K = 5

Context

Metrics

1

★ 0.726

SRC: ARS.pdf

namely, static analysis and dynamic analysis. This work will only use static approach for malware detection because static analysis is more effective than dynamic analysis. And,...

2

★ 0.608

SRC: ARS.pdf

modifications are in the system. Static analysis is a commonly used tool in malware detection. For Java applications, static analysis works directly on the bytecode and...

3

★ 0.570

SRC: ForcedBytecodePatching.pdf

may decrease a program analysis performance, and obscure the results if the analysis was performed using the IL with inadequate quality and reliability [6]–[8]. In...

AVERAGE SIMILARITY

0.63

How it's calculated: Mean Cosine Similarity (0 to 1) of all retrieved chunks.

Why it's relevant: High scores mean chunks are semantically very close to your exact query subject.

FAITHFULNESS

0.67

How it's calculated: Ratio of words in the LLM answer that directly overlap with words in the retrieved chunks.

Why it's relevant: Low faithfulness implies the LLM might be hallucinating outside the loaded context.

RELEVANCE

100%

How it's calculated: Percentage of retrieved chunks passing a strict similarity threshold (≥ 0.61).

Why it's relevant: Indicates if the retrieval hit high-quality paragraphs or just loosely related filler text.

COMPUTE LATENCY

0.03s R / 143.58s G

How it's calculated: Wall-clock time required for FAISS vector search and Local LLM token generation.

Why it's relevant: Monitors local hardware speeds and chunking performance regressions.



How Does It Work?

1. **Ingestion:** Documents and web links are processed and split into chunks using LangChain of exactly 750 characters.
2. **Embedding:** Those string chunks are mapped into semantic multidimensional vectors via the `all-MiniLM-L6` transformer.
3. **Organization:** FAISS plots the knowledge base in memory, allowing users to toggle specific sources active or inactive.
4. **Inference:** The selected Top-K chunks are injected alongside the user query to the local `TinyLlama` instance to compile an intelligent, cited reply.

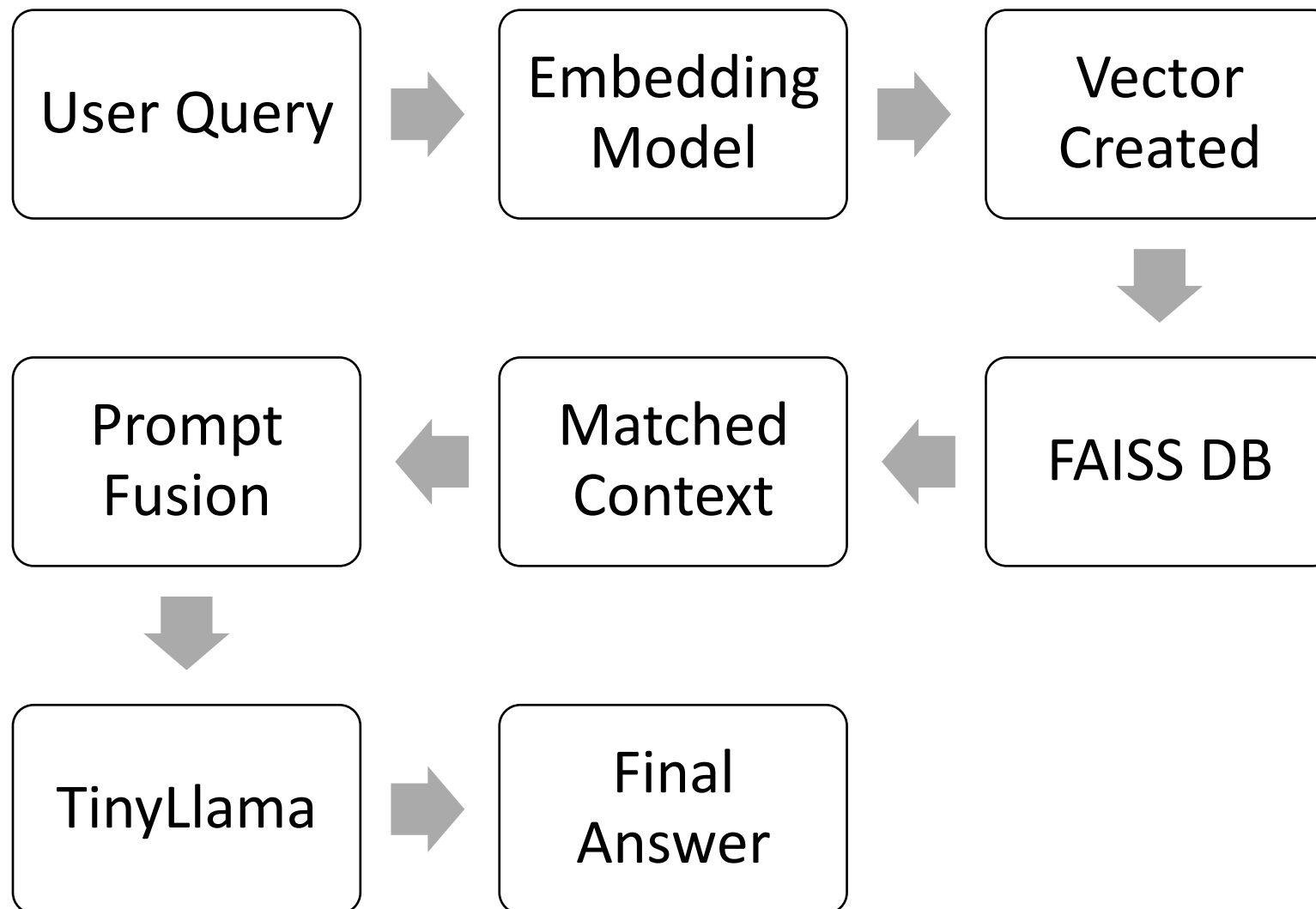


What It Can Be Used For

- **Academic/Research:** Rapidly processing large PDF textbooks logically for specific citations without reading 500 pages.
- **Development/Engineering:** Feeding complex API documentation URLs into an agent for instant coding context.
- **Legal/Enterprise:** Chatting securely with sensitive, confidential documents knowing the firewall remains uncompromised.



Flowchart





Evaluation Metrics

- **Avg Similarity:** The average mathematical "closeness" between your question and the retrieved chunks. High scores (0.7+) mean the system found very relevant text.
- **Faithfulness (Groundedness):** Measures how much of the answer is actually based on the context. It detects "Hallucination"—higher scores mean the model isn't making things up.
- **Relevance** The percentage of retrieved chunks that exceeded the system's "utility threshold". 100% means every piece of info pulled was high-quality.
- **Gen Latency:** The total time taken (in seconds) for the model to "think" and generate the final text reply on your local machine.



RESULTS AND ANALYSIS

Query: Explain the difference between static and dynamic malware analysis.

K = 1

Static malware analysis is a technique that analyzes the code and functionality of an application without executing it. This approach relies on identifying known patterns and behaviors within the program, which can be used to identify potential vulnerabilities or weaknesses.

On the other hand, dynamic malware analysis involves running the software under test in real-time to monitor its behavior and performance, which helps in detecting malicious activities such as stealing sensitive data, accessing protected resources, or disabling antivirus software.

In static analysis, the code is first compiled into machine code, which may affect the execution speed and memory usage. However, this approach ensures that the software will perform correctly at runtime, but requires additional overhead and processing power. In contrast, dynamic analysis can provide more accurate results due to the availability of the operating system's virtualization technology to run the code in a simulated environment.

This distinction can help security professionals determine the most effective approach when analyzing malicious software, taking into account factors such as resource constraints, budget, and the complexity of the analysis.

K = 3

Sure! Here's a brief explanation of the difference between static and dynamic malware analysis:

1. Static Malware Analysis:

Static malware analysis involves analyzing the compiled and finalized version of an executable file (e.g., .exe) at compile time. The analysis starts with a pre-compiled binary file (e.g., .o), which contains the instruction sets and symbols of the program. During this process, the analysis tools examine the assembly code, including the function calls and variable values. 2. Dynamic Malware Analysis:

Dynamic malware analysis involves analyzing running applications during runtime. This analysis is done by examining the memory, registers, and other resources of the system. During this process, the tools collect and analyze dynamic information such as object pointers, symbol tables, and function calls. Dynamic analysis detects malware by monitoring the behavior of the program during execution, while static analysis focuses on identifying potential vulnerabilities during compilation. Therefore, dynamic analysis is more efficient and cost-effective compared to static analysis in malware detection due to the availability of real-time information on the running system.

K = 5

Malware analysis refers to the process of identifying, understanding, and preventing malicious software from entering a computer system. There are three primary types of malware analysis techniques: static analysis, dynamic analysis, and hybrid analysis. Here is a brief explanation of each technique:

1. Static Analysis: Static analysis involves running the malware program on a compiled or interpreted binary program. The program is then analyzed to identify code patterns, functions, and variables used by the malware. These patterns are compared with those found in known malware samples to determine whether they match. Static analysis does not require execution of the malware program, making it less invasive than other approaches.

2. Dynamic Analysis: Dynamic analysis is a technique that analyzes the behavior of the malware program during execution. Dynamic analysis looks at the malware program's interactions with the operating system, network, and hardware to understand how the malware operates in real-time. Dynamic analysis requires execution of the malware program and can provide deeper insights into the malware's operation.

3. Hybrid Analysis: Hybrid analysis combines the strengths of both static and dynamic analysis techniques to obtain a complete picture of the malware's behavior. Hybrid analysis uses a combination of static and dynamic analysis to identify the malware's code structure and behavior during runtime.

Static analysis focuses on finding code patterns and functions that are common among malware samples. Dynamic analysis focuses on understanding the malware's behavior during execution, while hybrid analysis combines the benefits of both techniques to obtain a comprehensive view of the malware's operation.



Evaluation Metrics

- **Avg Similarity:** The average mathematical "closeness" between your question and the retrieved chunks. High scores (0.7+) mean the system found very relevant text.
- **Faithfulness (Groundedness):** Measures how much of the answer is actually based on the context. It detects "Hallucination"—higher scores mean the model isn't making things up.
- **Relevance** The percentage of retrieved chunks that exceeded the system's "utility threshold". 100% means every piece of info pulled was high-quality.
- **Gen Latency:** The total time taken (in seconds) for the model to "think" and generate the final text reply on your local machine.



RESULTS AND ANALYSIS

K=1

K=3

K=5

# 1 ★ 0.726 SRC: ARS.pdf namely, static analysis and dynamic analysis. This work will only use static approach for malware detection because static analysis is more effective than dynamic analysis. And,...	# 1 ★ 0.726 SRC: ARS.pdf namely, static analysis and dynamic analysis. This work will only use static approach for malware detection because static analysis is more effective than dynamic analysis. And,...	# 1 ★ 0.726 SRC: ARS.pdf namely, static analysis and dynamic analysis. This work will only use static approach for malware detection because static analysis is more effective than...
	# 2 ★ 0.608 SRC: ARS.pdf modifications are in the system. Static analysis is a commonly used tool in malware detection. For Java applications, static analysis works directly on the bytecode and...	# 2 ★ 0.608 SRC: ARS.pdf modifications are in the system. Static analysis is a commonly used tool in malware detection. For Java applications, static analysis works directly on the bytecode an...
	# 3 ★ 0.570 SRC: ForcedBytecodePatching.pdf may decrease a program analysis performance, and obscure the results if the analysis was performed using the IL with inadequate quality and reliability [6]–[8]. In...	# 3 ★ 0.570 SRC: ForcedBytecodePatching.pdf may decrease a program analysis performance, and obscure the results if the analysis was performed using the IL with inadequate quality and reliability [6]–[8]. In...
		# 4 ★ 0.543 SRC: ForcedBytecodePatching.pdf testing, clones, "bad smells", data and privacy leaks detec- tion, "points-to" analysis, and others. In dynamic program

Chunks Usage



RESULTS AND ANALYSIS

- **Similarity Decreases**

As K increases ($1 \rightarrow 5$), similarity drops ($0.73 \rightarrow 0.68$) because less relevant documents are also included.

- **Better Coverage with Higher K**

Higher K retrieves more information, so it reduces the chance of missing important details—but may add some irrelevant content.

- **Latency Stays Similar**

Response time doesn't change much because the main limitation is CPU processing, not the number of documents.

- **Best Value of K**

$K = 3$ or 4 gives a good balance between accuracy and completeness.



RESULTS AND ANALYSIS

Metric	K = 1	K = 3	K = 5
Avg Similarity	0.73	0.70	0.68
Faithfulness	0.83	0.78	0.81
Relevance	100%	100%	100%
Gen Latency (s)	230.53	229.71	235.24



Thank You